APOLLO in a Reduced Setting: Memory-Efficient Adaptive Updates for Small Transformer Pre-Training

Author Name

March 19, 2026

Abstract

This report studies the central idea behind APOLLO: adaptive updates for Transformer pre-training contain substantial redundancy, so the element-wise scaling rule of AdamW can be coarsened and then approximated in a low-rank subspace without a large loss in training quality. Our goal is not to reproduce the full MLSys-scale setup of the paper, but to verify the claim in a reduced GPT-style setting with public datasets and limited compute. In particular, we focus on the trade-off between optimizer-state memory and training quality, and ask whether APOLLO-like updates can reduce optimizer memory substantially while remaining close to AdamW in validation loss.

1 Introduction

Why optimizer memory matters. AdamW remains the default optimizer for Transformer training because its adaptive first- and second-moment statistics stabilize optimization and usually outperform purely first-order methods such as SGD on language models [1, 2, 3]. However, this stability comes with a substantial memory cost. For a parameter tensor W_t with gradient $G_t = \nabla_W \mathcal{L}(W_t)$, AdamW maintains two full-size state tensors, the first moment M_t and the second moment V_t:

$$M_t = \beta_1 M_{t-1} + (1-\beta_1) G_t, \tag{1}$$

$$V_t = \beta_2 V_{t-1} + (1-\beta_2) G_t^2, \tag{2}$$

$$W_{t+1} = W_t - \eta \cdot \frac{M_t}{\sqrt{V_t} + \varepsilon}. \tag{3}$$

Ignoring weight decay, AdamW therefore stores two extra tensors with the same shape as the parameters. If these optimizer states are stored in FP32, the memory overhead is

$$\text{AdamW optimizer states} = 2P \times 4 \text{ bytes} = 8P \text{ bytes}, \tag{4}$$

for a model with P parameters. In our reduced setup, the current GPT-style model has $P \approx 40.7$M parameters, so AdamW moments alone require about

$$8P \approx 3.26 \times 10^8 \text{ bytes} \approx 0.30 \text{ GiB}. \tag{5}$$

For comparison, a 124M-parameter model would already require about 0.92 GiB just for the two moment buffers. This is precisely the scaling problem emphasized by APOLLO: as models grow, optimizer states quickly become one of the dominant memory terms [7].

From element-wise scaling to structured scaling. APOLLO starts from a simple but important observation: AdamW can be reinterpreted as an element-wise gradient scaling rule rather than only as a momentum-based optimizer. Define the AdamW-scaled gradient

$$\widetilde{G}_t = \frac{M_t}{\sqrt{V_t} + \varepsilon}. \tag{6}$$

Then AdamW is equivalent to

$$W_{t+1} = W_t - \eta \cdot \widetilde{G}_t = W_t - \eta \cdot S_t \odot G_t, \tag{7}$$

where $S_t = \widetilde{G}_t \oslash G_t$ is an element-wise scaling tensor [7]. The core hypothesis of the paper is that this scaling rule is often more fine-grained than necessary. Instead of assigning one scale per parameter, one may share a scale across an entire channel or even an entire tensor:

$$\widetilde{G}_t^{\text{ch}} = G_t \cdot \operatorname{diag}(s_t), \qquad s_t[j] = \frac{\left\|\widetilde{G}_t[:, j]\right\|_2}{\left\|G_t[:, j]\right\|_2}, \tag{8}$$

for channel-wise scaling, and

$$\widetilde{G}_t^{\text{tensor}} = s_t^{\text{tensor}} G_t, \qquad s_t^{\text{tensor}} = \frac{\left\|\widetilde{G}_t\right\|_2}{\left\|G_t\right\|_2}, \tag{9}$$

for tensor-wise scaling. In other words, the structured update keeps the raw gradient direction but replaces the element-wise adaptive scale with a coarser channel-level or tensor-level factor.

Is structured scaling already enough? The paper first validates this idea empirically by comparing element-wise and channel-wise learning-rate adaptation on a LLaMA-130M model, showing that channel-wise adaptation reaches a very similar training-loss trajectory and competitive validation perplexity [7]. This part of the story is conceptually close to recent work on optimizer redundancy, especially Adam-mini, which argues that block- or group-wise second-moment statistics can be sufficient for Transformer training [4]. More broadly, the motivation is consistent with recent studies suggesting that Transformers do not always require a fully independent adaptive scale for every parameter [3].

That said, structured scaling by itself does *not* solve the memory problem. To compute s_t exactly, we still need access to the full AdamW moments M_t and V_t. Therefore, a pure channel-wise or tensor-wise reformulation is mainly a *mechanistic* observation: it suggests that the update rule can be coarsened without obvious optimization failure, but it does not yet eliminate the full optimizer-state memory.

Why structured updates alone do not save memory. This distinction is important for our reduced experiments. A naive implementation of channel-wise scaling still materializes the full AdamW moments, computes the full element-wise update, and then compresses it into a channel-level scale. In that case, the persistent optimizer states remain full rank, so the method inherits nearly all of AdamW's state memory. In practice, peak memory may even increase because the structured optimizer step introduces additional temporary tensors. This is exactly why APOLLO does not stop at structured scaling: the paper explicitly states that coarsening the learning-rate rule does not inherently reduce optimizer memory, since one still needs the full M_t and V_t tensors to compute the scales [7].

From structured scaling to low-rank subspace updates. The practical contribution of APOLLO is to combine two ideas:

1. the adaptive scale can be shared across a channel or a whole tensor, and
2. the statistics needed to estimate this scale can be maintained in a low-rank subspace.

Low-rank gradient optimization was already explored by methods such as GaLore and Fira, which reduce optimizer-state memory by updating projected gradients instead of full gradients [5, 6]. The main drawback is that these methods commonly rely on singular value decomposition (SVD) to build the subspace, and repeated SVD updates are expensive. APOLLO replaces this step with random projection:

$$R_t = P_t G_t, \qquad P_t \in \mathbb{R}^{r\times m}, \tag{10}$$

and stores low-rank moments M_t^R and V_t^R in the projected space:

$$M_t^R = \beta_1 M_{t-1}^R + (1-\beta_1) R_t, \tag{11}$$

$$V_t^R = \beta_2 V_{t-1}^R + (1-\beta_2) R_t^2. \tag{12}$$

The resulting channel-wise scale is then estimated from the low-rank statistics instead of the full-rank ones:

$$s_t^R[j] = \frac{\left\|\widetilde{R}_t[:,j]\right\|_2}{\|R_t[:,j]\|_2}, \qquad \widetilde{R}_t = \frac{M_t^R}{\sqrt{V_t^R}+\varepsilon}. \tag{13}$$

This changes the optimizer-state memory from $2mn$ to roughly $2nr$ for a matrix $W \in \mathbb{R}^{m\times n}$ with $m \le n$ [7]. In the extreme tensor-wise rank-1 case of APOLLO-Mini, the memory cost becomes comparable to SGD-level state storage.

Why random projection should still work. The paper's theoretical support is not a generic convergence theorem for the entire optimizer, but rather a sequence of approximation guarantees showing that random projection preserves the key norms needed to estimate the structured scaling factors. The analysis argues that if the projected gradients preserve channel-wise first- and second-moment norms sufficiently well, then the resulting low-rank scaling factors remain close to those of the full-rank structured optimizer [7]. This is the key justification for replacing SVD with random projection: APOLLO does not try to reconstruct the full gradient exactly, but only the coarse adaptive scaling needed to modulate the gradient update.

Our claim in a reduced setting. The original paper evaluates APOLLO on LLaMA models up to 7B parameters and C4 pre-training, which is outside the scope of a small class project. Our goal is therefore narrower: in a reduced GPT-style setup on public datasets and a much smaller model, we test whether an APOLLO-like optimizer can substantially reduce optimizer-state memory while keeping training quality reasonably close to AdamW. We also use the reduced setting to separate three logically distinct questions:

1. Is element-wise adaptive scaling redundant enough that channel-wise or tensor-wise scaling remains viable?
2. Does structured scaling alone save memory in practice?
3. Does low-rank random projection provide the missing step that converts structured scaling into a genuinely memory-efficient optimizer?

This decomposition is important for interpreting our experiments: pure structured updates are best viewed as a sanity check for the *mechanism*, while APOLLO is the part that turns this mechanism into a memory-efficient training method.

2 Experimental Setup

To be completed. This section will describe the reduced model setup, datasets, optimizers, metrics, and the exact claims tested.

3 Results

To be completed. This section will include training curves, validation curves, memory breakdowns, and the comparison between AdamW and APOLLO-style updates.

4 Discussion

To be completed. This section will discuss agreement and disagreement with the original paper, especially the difference between structured scaling as a hypothesis and APOLLO as a practical optimizer.

References

[1] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[2] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. *International Conference on Learning Representations*, 2019.

[3] Yuxin Zhang, Chen Chen, Ting Ding, et al. Why Transformers Need Adam: A Hessian Perspective. *arXiv preprint arXiv:2402.16788*, 2024.

[4] Zhen Zhang, Hanqing Lu, et al. Adam-mini: Use Fewer Learning Rates to Gain More. *arXiv preprint*, 2024.

[5] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, et al. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. *ICML*, 2024.

[6] Xinjie Chen, Kexin Feng, Chengyuan Li, et al. Fira: Can We Achieve Full-Rank Training of LLMs Under Low-Rank Constraint? *arXiv preprint*, 2024.

[7] David Z. Pan, Hanqing Lu, Zhangyang Wang, and Jinwon Lee. APOLLO: SGD-like Memory, AdamW-level Performance. *Proceedings of MLSys*, 2025.